Linked List Tutorial

A **linked list** is an extremely important topic for interviews and is frequently asked in companies like Facebook (Meta), Microsoft, Amazon, and Google. Unlike arrays, which generally have continuous memory allocation, a linked list does not store its items in contiguous memory. Instead, it breaks down items into separate **nodes**, which can be placed anywhere in memory.

---

1. Limitations of Arrays (addressed by Linked Lists)

- **Fixed Size**: If an array reaches its fixed size, you cannot add new items. While `ArrayList` can dynamically resize, this involves copying elements (though amortized O(1)).
- **Memory Allocation**: Arrays require continuous memory, which can be restrictive.

---

2. Advantages of Linked Lists

- **Dynamic Size**: Items can be added without worrying about fixed capacity.
- **Non-Contiguous Memory**: Nodes can be stored in different places in memory.

---

3. Key Concepts in Linked Lists

- **Node**: Each element in a linked list.
- **Value**: Data stored in the node.
- **Next Pointer**: Reference to the next node in the sequence.
- **Head**: Points to the first node.
- **Tail**: Points to the last node (useful for O(1) insertions at the end).

General Approach for Linked List Problems: 1. Draw the initial state. 2. Visualize the final state. 3. Identify which pointers need updating. 4. Translate into code by modifying `next` (and `previous` in DLL).

Important Note: Never move the `head` during traversal. Use a temporary pointer instead.

---

4. Types of Linked Lists

- **Singly Linked List**
- **Doubly Linked List**
- **Circular Linked List**

---

5. Singly Linked List

A **singly linked list** represents a one-sided relationship. Each node only knows about the next node.

Node Structure

```
class Node {
    int value;
    Node next;
}
```

Internal Implementation

The `LinkedList` class usually contains: - `Node head` - `Node tail` - `int size`

Operations

Insert First (O(1))

1. Create newNode.
2. Set `newNode.next = head`.
3. Update `head = newNode`.
4. If list empty, `tail = head`.
5. Increment size.

Insert Last (O(1) with tail, O(N) without tail)

1. Create newNode.
2. `tail.next = newNode`.
3. Update `tail = newNode`.
4. If empty, call `insertFirst()`.
5. Increment size.

Insert at Index (O(N))

1. If `index == 0`, call `insertFirst()`.
2. If `index == size`, call `insertLast()`.
3. Otherwise, traverse to index-1.
4. Insert newNode between `current` and `current.next`.

Display (Traversal) (O(N))

1. Use `temp = head`.
2. While `temp != null`, print `temp.value`.
3. Move `temp = temp.next`.

Delete First (O(1))

1. Store head value.
2. Update `head = head.next`.

3. If head == null, set tail = null.
4. Decrement size.

Delete Last (O(N))

1. If `size <= 1`, call `deleteFirst()`.
2. Traverse to `size-2`.
3. Set `tail = secondLast`.
4. `tail.next = null`.

Delete at Index (O(N))

1. If `index == 0`, call `deleteFirst()`.
2. If `index == size-1`, call `deleteLast()`.
3. Traverse to `index-1`.
4. Remove `previous.next`.

Find a Value (O(N))

1. Traverse from head.
2. If `temp.value == target`, return node.
3. Else return null.

---

6. Doubly Linked List

A **doubly linked list (DLL)** allows traversal both forward and backward.

Node Structure

```
class Node {
    int value;
    Node next;
    Node previous;
}
```

Operations

Insert First (O(1))

1. Create newNode.
2. `newNode.next = head`.
3. `newNode.previous = null`.
4. If head != null, `head.previous = newNode`.
5. Update `head = newNode`.

Display Forward (O(N))

Traverse using `node = node.next`.

Display Reverse (O(N))

1. Traverse to last node.

2. Traverse backward using `node = node.previous`.

Insert Last (O(N) without tail, O(1) with tail)

1. Create newNode.
2. If empty, set as head.
3. Else traverse to last.
4. `last.next = newNode`.
5. `newNode.previous = last`.

---

7. Circular Linked List

A **circular linked list** connects the last node back to the head.

Node Structure

```
class Node {
    int value;
    Node next;
}
```

Insertion (O(1))

1. Create newNode.
2. `tail.next = newNode`.
3. `newNode.next = head`.
4. Update `tail = newNode`.
5. If empty, head = tail = newNode (points to itself).

Display (O(N))

1. Use a do-while loop.
2. Start from head.
3. Print until reaching head again.

Deletion (O(N))

- **Delete Head**: Move head = head.next, update tail.next = head.
- **Delete Value**: Traverse until found, then bypass node.

---

8. Key Takeaways

- Arrays are static and require contiguous memory.
- Linked lists are dynamic and use scattered memory.
- Singly: forward traversal.
- Doubly: forward + backward.

- Circular: continuous loop.
- Always visualize pointer changes before coding.